When Do Dense Process Rewards Help Agentic Reinforcement Learning? A Verifiable-Potential Criterion

Bojie Li
Pine AI

Abstract

Agentic reinforcement learning inherited its reward from reasoning models: a single bit at the end of a trajectory saying whether the task was solved. This is doubly wasteful. On long-horizon tasks the base policy rarely succeeds, so most sampled groups are *all-fail*, and group-relative methods such as GRPO assign them an advantage of exactly zero—no gradient—during precisely the phase of training when the policy most needs shaping. And an agent's environment is itself a *verifier*: every tool call returns a machine-checkable result, a dense intermediate signal that reasoning RL never had and that outcome-only RL discards. The obvious fix—reward the process—is treacherous: a penalty an agent can satisfy by doing nothing drives it into a degenerate *compliance* optimum, and a learned progress critic is simply hacked. We ask precisely *when* a dense process reward helps, and answer with a criterion: **a process reward improves agentic RL if and only if it is an un-gameable, verifiable proxy for progress—a potential strictly finer than the terminal outcome whose intermediate values the policy can actually reach.** Whether such a signal exists is a property of the *domain*, not the algorithm.

We turn the criterion into a test applied *before* training and validate it as a predictive law across five settings spanning both verdicts. Where it says *yes*: in theorem proving a verifiable progress signal turns the all-fail groups into gradient and eliminates dead updates entirely; in system administration an un-gameable harm penalty cuts destructive actions roughly fourfold at *equal* task success—reducing harm on the path that the terminal outcome cannot see. Where it says *no*: in customer-service workflows the residual difficulty is task *intent*, which no verifiable rule can encode, so process rewards neutralize harm but cannot beat outcome-only; in software repair the finer signal exists for only a third of problems and is *empirically unreachable*—the policy never makes partial progress—so it supplies no gradient at all. A controlled sweep that walks a single signal from aligned to misaligned reproduces the criterion's verdict arm by arm: signals carrying a misaligned penalty collapse, penalty-free progress signals survive, and gating a gameable signal on the outcome restores it. The criterion explains our gains and our failures with one rule, and tells a practitioner, before spending any compute, whether a dense process reward is worth adding.

1 Introduction

Outcome-only reinforcement learning—sample a group of trajectories, reward the ones that solve the task, push the policy toward them with a group-relative advantage—carried reasoning models from a base checkpoint to sophisticated behaviour with no supervised cold-start [DeepSeek-AI, 2025, Shao et al., 2024], and the agentic era inherited it wholesale: a

tool-using agent is trained by rewarding the trajectories that complete the task, and nothing else. In production this leaves a stubborn gap—a ByteDance keynote reports frontier coding agents are functionally correct over 80% of the time yet score far lower, and with high variance, on the engineering qualities that decide whether code ships: reliability, reuse, maintainability [Hong, 2026].

But an agentic trajectory is not a chain of thought. It is a sequence of *interactions with an environment*, and the environment is itself a *verifier*: error codes, tool results, and state transitions are machine-checkable intermediate signal that reasoning RL never had—you cannot verify a chain-of-thought step mid-stream without redoing it. Outcome-only RL discards this signal, and pays twice. The first cost is *gradient*, and it is the lens we use throughout: a group-relative advantage is, mechanically, a within-group variance—GRPO centres each reward on the group mean [Shao et al., 2024], so a group can teach the policy only insofar as its rewards *differ*. Early in training on a hard task most groups are *all-fail*—every rollout misses—and a binary outcome has zero variance there: identical rewards, zero advantage, a dead update, exactly when the policy most needs shaping. A dense, verifiable *process* signal restores the variance by distinguishing the failed rollouts the outcome cannot tell apart. This is the paper in one sentence—**a useful dense process reward is the within-group variance the outcome cannot supply**—and most of what follows is establishing when such a signal exists, and when, used naively, it backfires. The second cost is *safety*: the outcome reward is defined on the terminal state, so it is blind to harm *on the path*—an agent that deletes a production database and then rebuilds it ends in a correct state and is rewarded, though the irreversible damage is already done.

The intermediate signal the environment provides could fill both gaps. The trouble is that using it naively backfires, in ways that have made practitioners wary of process rewards altogether. A penalty an agent can satisfy by *doing nothing*—"never call before looking up the account"—is maximized by an idle policy, and in the all-fail regime, where the process channel is the only gradient, the optimizer climbs straight into that degenerate *compliance* optimum and task success collapses to zero. And a *learned* progress critic, the natural way to get a dense signal without writing rules, simply moves the reward-hacking up one level: the policy learns to fool the critic. Process rewards sometimes give large gains and sometimes cause catastrophic collapse, and the field has lacked a way to tell the two cases apart in advance.

This paper. We give that way. We argue that whether a dense process reward helps is not a property of the algorithm but of the *domain*, and we make it precise with a single criterion (Figure 1):

> *A dense process reward improves agentic RL if and only if it is an **un-gameable, verifiable proxy for progress**: a potential strictly finer than the terminal outcome (so it varies across the failed rollouts in an all-fail group), whose cheapest maximizer must still solve the task (so the optimizer cannot game it), and whose intermediate values the policy can **actually reach** (so the variation is realized, not merely possible).*

The three conditions—finer-than-outcome, un-gameable, reachable—are each individually testable *before* training, by inspecting the domain and a handful of base-policy rollouts. The criterion is grounded in potential-based reward shaping, which guarantees that any potential leaves the optimal policy unchanged [Ng et al., 1999]: a process reward is therefore safe

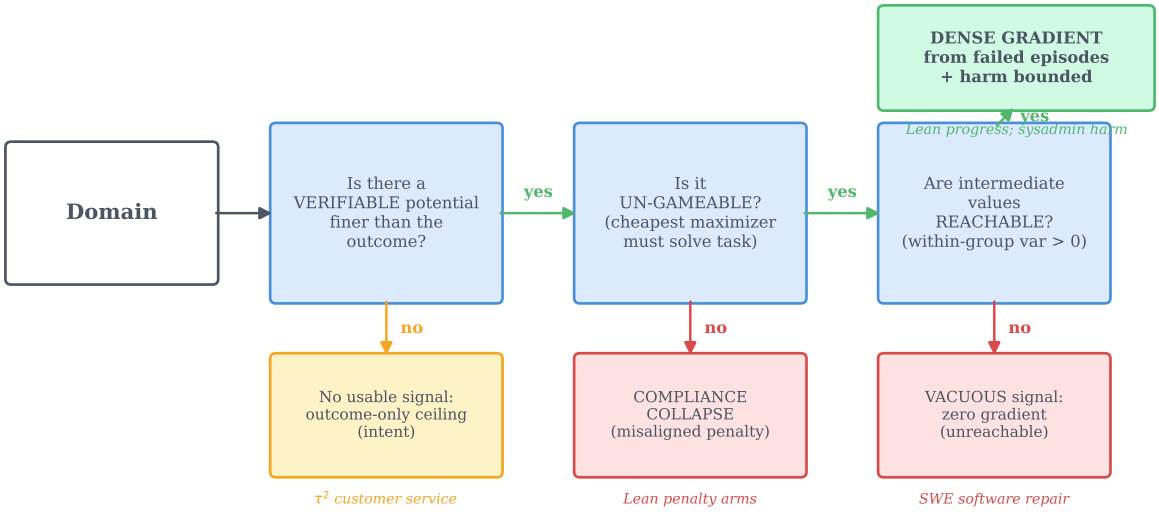

Figure 1. The verifiable-potential criterion, and the five settings it sorts. Reading left to right, a dense process reward helps only if the domain passes three gates: a verifiable potential *finer* than the terminal outcome must exist, it must be *un-gameable* (its cheapest maximizer still has to solve the task), and its intermediate values must be *reachable* by the policy. Failing the first gate leaves only the outcome ceiling (customer-service *intent*); failing the second invites compliance collapse (a misaligned penalty); failing the third makes the signal vacuous (software repair, where partial progress never occurs). Passing all three yields dense gradient from failed episodes and a bounded harm path (theorem proving; system administration).

to add exactly when it is such a potential, and useful exactly when it additionally supplies group-relative gradient the outcome cannot.

Validation as a predictive law. We test the criterion not by proposing one method and showing it works, but by treating our entire body of results—successes *and* failures—as predictions the criterion must get right. Across five settings (Table 1), the criterion's verdict matches the measured outcome in every case. Where it says *yes*: in theorem proving a kernel-verified progress signal turns all-fail groups into gradient and eliminates dead updates; in system administration an un-gameable harm penalty cuts destructive actions roughly fourfold at *equal* task success. Where it says *no*: in customer-service workflows the residual difficulty is task *intent*, which is the reward itself and not any verifiable rule, so process rewards neutralize harm but never beat outcome-only; in software repair a finer potential exists for only a third of problems and is empirically *unreachable*, so it supplies no gradient. A controlled sweep that walks one signal from aligned to misaligned then reproduces the criterion's verdict arm by arm within a single experiment.

Contributions.

- **A criterion** (§2) that predicts, before training, whether a dense process reward will help in a given domain—unifying the all-fail-group blindness of group-relative RL, the compliance attractor of misaligned penalties, and the failure of learned critics under one condition: an un-gameable, reachable, verifiable potential finer than the outcome.
- **A controlled demonstration that the criterion is predictive** (§3): a three-seed un-gameability

sweep at 30B walks a single Lean signal from aligned to misaligned and the policy's fate tracks admissibility—penalty-free progress signals survive, a pure penalty reliably collapses, adding a discharge to the penalty makes survival possible but high-variance, and outcome-gating a gameable signal gives the most consistent survivor; a granularity/sparsity study shows the gain scales with the potential's fineness; and an SWE reachability probe shows a structurally-finer potential that is empirically vacuous.

- **Two positive instantiations** where the criterion says yes: process rewards make outcome learning *sample-efficient* by converting dead updates into gradient (§4–5), and an un-gameable penalty achieves *harm reduction on an axis the outcome cannot see* (§6).
- **Three boundaries** where it says no, mapped honestly: an intent ceiling on a real benchmark where verifiable rules cover policy *validity* but not task *intent* (§7); a learned self-critic shown to relocate rather than remove the problem (§8); and a discovery ceiling where no on-policy reward escapes a never-sampled precondition (§9).

2 A Criterion for When Process Rewards Help

The gradient gap (why one would want a process reward). Group-relative policy optimization centers each trajectory's reward on its group mean. When rewards vary within a group the centering is informative; when *every* rollout fails, the rewards are identical, every advantage is exactly zero, and the group yields no gradient—a *dead update* (Figure 2). On short-horizon tasks a competent base model makes all-fail groups rare. On long-horizon agentic tasks the base success rate is low, all-fail groups dominate early training, and outcome-only RL is starved of signal during exactly the phase that determines whether it ever gets off the ground. A process reward that depends on *how* an episode behaves rather than *whether* it succeeds is non-zero on those very groups—if it varies across the failed rollouts, it is gradient where the outcome has none.

The two failure modes (why one cannot use just any process reward). The same all-fail regime that makes a process reward valuable also makes it dangerous, because the process channel is then the *only* gradient. If the cheapest way to maximize the process reward is something other than solving the task, the optimizer will find it. Two cases recur. A **penalty** is almost always gameable: its cheapest maximizer is inaction (never act $\Rightarrow$ never violate), so on all-fail batches it drives the policy into a degenerate compliance-only optimum—success goes to zero while "compliance" is perfect. A **learned critic** is gameable in a subtler way: defining progress by a learned proxy relocates the credit-assignment problem into the proxy, which the policy then learns to fool. Neither failure is about tuning; both are structural.

The criterion. These observations have a common root. A process reward is admissible as a learning signal precisely when it is an **un-gameable verifiable potential finer than the outcome**, and it is *useful* when, in addition, the policy reaches its intermediate values. Concretely, three conditions, each checkable before training:

(1) **Finer than outcome.** The signal Φ takes more values than the binary terminal reward, and those values *vary across the rollouts in an all-fail group*. (A uniform Φ is centered out by GRPO and supplies no gradient.)

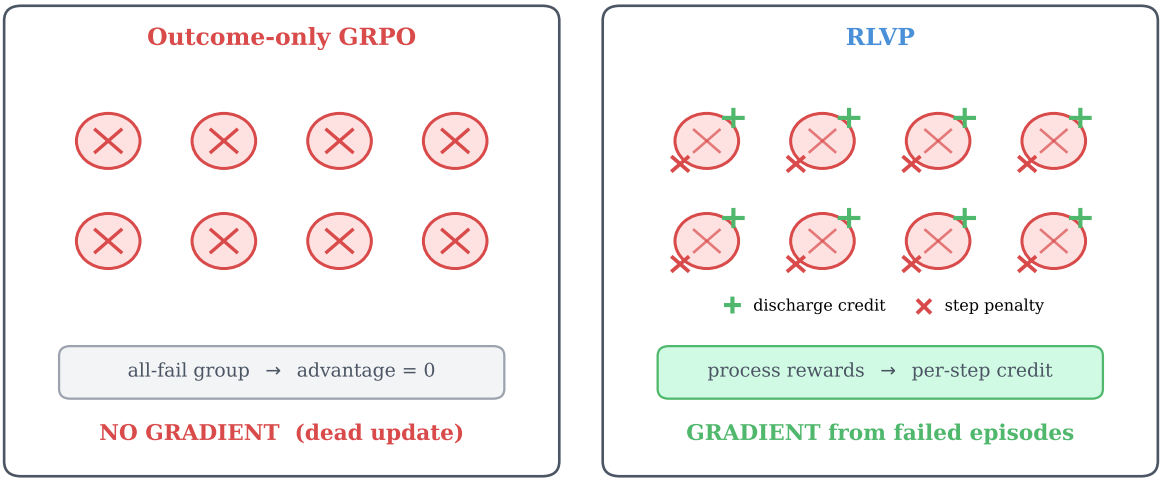

Figure 2. Why a process signal can help where the outcome cannot. On a hard task most early groups are *all-fail*. *Left:* outcome-only GRPO sees identical (zero) rewards, so the group-mean-centered advantage is zero on every token—a dead update. *Right:* a process signal attached to individual tool calls differs across the failed episodes (one read before writing, another did not), so the update is non-zero. The condition for this to work is that the process signal *varies within the group*—i.e. it is a potential strictly finer than the outcome.

(2) **Un-gameable.** The cheapest policy that maximizes Φ must still solve the task. Operationally: name the cheapest policy that maximizes Φ *without* solving the task; if it is degenerate (idle, padding, error-avoidance by inaction), Φ is inadmissible.

(3) **Reachable.** The policy's rollouts actually attain intermediate Φ values—there is realized within-group variance, not merely a potential that exists on paper.

These three conditions are not independent; they are the three terms of one quantity. Writing each rollout's reward as its outcome plus a weighted potential, $R_i = O_i + \beta\,\Phi_i$, the only thing a group-relative method extracts from a group is the spread of its rewards,

$$\mathrm{Var}_G(R) \;=\; \mathrm{Var}_G(O) \;+\; \beta^2\,\mathrm{Var}_G(\Phi) \;+\; 2\beta\;\mathrm{Cov}_G(O,\Phi),$$

and the gates are its terms: the outcome term vanishes on all-fail groups; the potential term is non-zero only when Φ is both finer than the outcome *and* reached—conditions (1) and (3), together $\mathrm{Var}_G(\Phi) > 0$; and the cross term, whether that variation points toward success, is what condition (2) protects. (The method's per-channel normalisation and per-token placement, §5, rescale and route this signal but leave which terms vanish, and the sign of the alignment, unchanged.) Potential-based shaping [Ng et al., 1999] certifies that any such Φ is *safe*—it leaves the optimal policy unchanged; the variance reading says which is *useful*. A verifiable progress credit (Lean's goal count strictly decreasing, a failing test turning green) can satisfy all three conditions; a bare penalty fails (2); a learned critic fails verifiability and usually (2); a domain whose only finer signal is unreachable fails (3). Table 1 previews how the five settings we study fall out.

Table 1. The criterion sorts five settings, and its verdict matches the measured outcome in every one. Each row is a domain; the three middle columns are the criterion's gates (does a verifiable potential finer than the outcome exist; is it un-gameable; are its intermediate values reachable). "✓/✗" marks the gate that decides the row. The criterion predicts *help* only when all three pass.

Setting	Domain / model	Finer Φ?	Un-game?	Reach?	Verdict	Observed
Theorem proving (progress)	Lean miniF2F 30B; synth. 4B	✓	✓	✓	**help**	dead updates $\to$ 0; dense gradient
System admin (harm)	TerminalBench 4B	✓	✓	✓	**help** (harm axis)	$\sim$4$\times$ fewer violations, equal success
Lean signal sweep (controlled)	miniF2F 30B	varies	**varies**	✓	per arm	pure penalty dies; penalty-free survive
Customer service (intent)	τ-bench airline 4B	✗	—	—	**no help**	rules match, never beat outcome
Software repair	SWE-bench 30B	1/3 only	—	✗	**no help**	0% solve; all rollouts Φ=0

3 The Criterion as a Predictive Law

If the criterion is more than a restatement of intuition, it should make *advance predictions* that a controlled experiment confirms. We provide three such tests.

An un-gameability sweep: admissibility predicts survival. We construct five process signals for Lean theorem proving on miniF2F [Zheng et al., 2022], spanning aligned to misaligned, and pre-register each one's cheapest gaming policy and the criterion's verdict. We then train Qwen3-30B-A3B [Team, 2025] with each, three seeds, holding everything else fixed (Figure 3). The result tracks admissibility, and three seeds sharpen exactly where the criterion is sharp and where it is not. The *penalty-free* signals reliably **survive** on every seed: a gameable "any-valid-tactic" credit (the most stable arm), an aligned goal-progress discharge, and that gameable credit *gated on the eventual outcome*—which is un-gameable by construction and gives the most consistent survivor. The *pure penalty* (an errored-tactic penalty with no discharge) reliably **collapses** on every seed to essentially zero: its cheapest maximizer, avoid errors by not attempting, is the compliance attractor, and with no positive signal to escape it the policy dies every time. The revealing case is the *penalty-plus-discharge* arm: single-seed it looked dead, but across three seeds it is *bimodal*—it collapses on two seeds and is rescued all the way to perfect success on the third, the largest variance of any arm. So adding a verifiable discharge to a misaligned penalty does not *reliably* save it; it makes survival possible but unstable. The robust law is therefore narrower and cleaner than a single seed suggested: **penalty-free progress signals survive, a pure penalty reliably collapses, and outcome-gating yields the most consistent survivor**—each called by the criterion in advance.

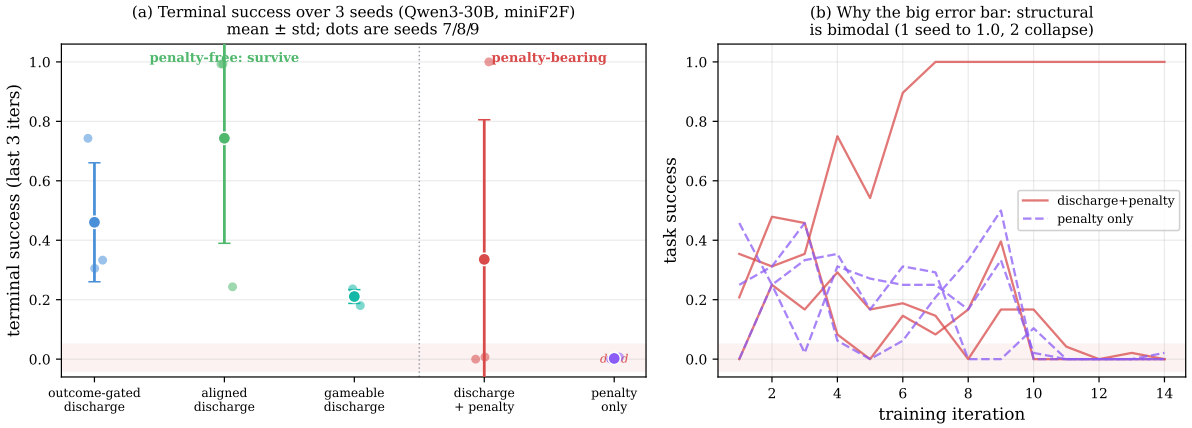

Figure 3. The criterion predicts which signals survive (Qwen3-30B, miniF2F, 3 seeds). *(a)* Terminal success (last-3-iteration mean) for five process signals, mean $\pm$ std with the three seed values as dots. The three *penalty-free* signals (left of the divider) sit well above the dead zone on every seed; the *pure penalty* (right) is reliably dead. The *discharge + penalty* arm has by far the largest error bar. *(b)* Why: that arm is *bimodal*—it collapses on two seeds but the goal-progress discharge rescues it to 1.0 on one—whereas the pure penalty collapses on all three. The robust reading is *penalty-free survives, pure penalty reliably collapses, gating gives the most consistent survivor*; the precise magnitudes remain noisy at 30B (§10).

Granularity and sparsity: the gain scales with the potential's fineness. A complementary, lower-variance test isolates condition (1). On a synthetic N-stage chained task with a verifiable potential equal to the number of completed stages, we vary the potential's *granularity* (coarse = outcome, versus one milestone, versus every stage) and the outcome's *sparsity* (N). A potential strictly finer than the outcome drives success where the outcome alone is essentially zero, and the advantage *grows* as the outcome becomes sparser—exactly the regime where all-fail groups dominate and the finer potential has the most uniquely-available gradient to offer. This is the positive, controlled face of the criterion: fineness is not incidental, it is the mechanism.

A reachability probe: a finer potential that is vacuous. Conditions (1) and (2) are about the signal; condition (3) is about the *policy*, and it is the one that quietly fails in hard domains. We test it directly on SWE-bench software repair [Jimenez et al., 2024], where the natural potential is the fraction of the hidden `FAIL_TO_PASS` tests an episode makes pass. Two facts emerge (Figure 4). Structurally, the finer potential barely exists: two-thirds of instances have a single failing test, so Φ collapses to the binary outcome; only a third are "multi-test" with any room for partial progress. Empirically, even that third is *unreachable*: across 156 rollouts of a 30B policy, *every* episode scores $\Phi = 0$—the model never makes a single hidden test pass, let alone a proper subset. The within-group variance is exactly zero, so a Φ-based reward would be centered out to no gradient. This is why the dense SWE signal fails, and the criterion locates the cause precisely: not gameability, but *unreachability*—there is no realized intermediate state for any reward, gameable or not, to act on.

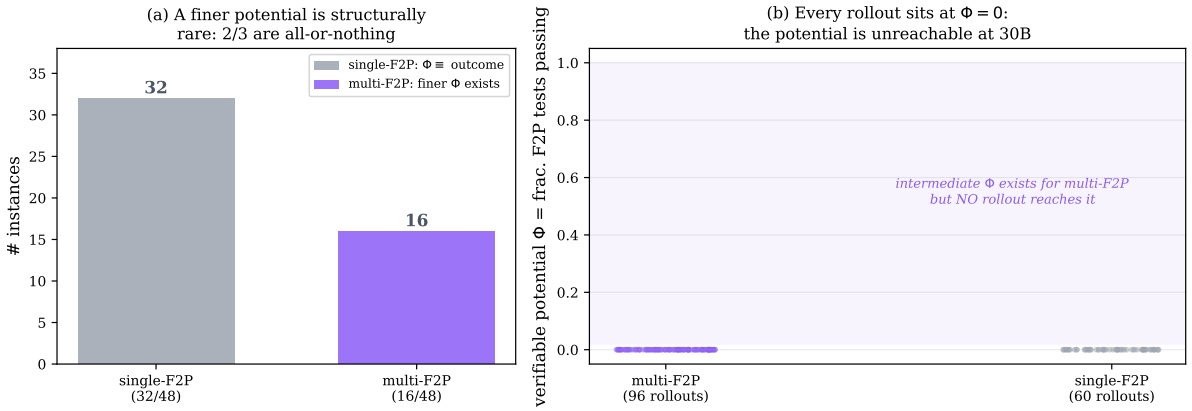

Figure 4. In software repair the finer potential is structurally rare and empirically unreachable. *(a)* Of 48 single-file-fix SWE-bench instances, two-thirds have a single failing test, so the test-fraction potential equals the binary outcome; only one-third admit a finer Φ. *(b)* For those, a 30B policy's rollouts never realize it: all 156 rollouts (multi- and single-test) score $\Phi = 0$. The shaded band of intermediate values exists but is empty—zero within-group variance, hence zero gradient. The criterion's third gate, reachability, is what fails here.

4 Where the Criterion Says Yes, I: Sample-Efficiency in Theorem Proving

When the verifiable signal *is* the bottleneck to success—read before write, decrease the goal count, pass the gate—the process channel and the outcome point the same way, and the criterion predicts a large efficiency gain. We confirm it on a controlled suite of long-horizon agentic tasks, where every claim about gradient is exact rather than estimated. The testbed is a family of *N-stage chained tasks*: an agent completes N independent file-operations subtasks in one episode using tools `list_dir`, `read_file`, `write_file`, `delete`, `run_tests`, `submit`; success requires all N stages, so N tunes base difficulty smoothly (N=4 sits near 8% base success). We instantiate the process signal as **Reinforcement Learning from Verifiable Penalties** (RLVP): a penalty when a call violates a verifiable rule (overwrite a file never read; submit without testing a change) and a credit when a call *discharges* a pending obligation (reads before writing, tests after mutating). These are *specifications*, written once, requiring no solved trajectories—the analog of R1-Zero's verifier, not of a supervised cold-start. The credit is carried on a separate channel with its own normalizer and attached to the *responsible tokens*; all experiments train Qwen3-4B [Team, 2025] with our own GRPO, and efficiency is always counted in *episodes generated* so that resampling costs are visible.

Dead updates are not a rounding error, and the fix is causal. We sample a batch of rollouts *once* and score it under both credit schemes (Figure 5). Outcome-only credit produces a dead, zero-gradient update on one in four identical batches; the process credit is never dead, because it produces gradient from the same failed episodes the outcome channel reduces to zero. Same rollouts, opposite outcomes: the effect is the credit scheme, not the data.

The gain is large and consistent; DAPO's is neither. Figure 6 plots held-out success against episodes generated. Outcome-only GRPO crawls—it spends most of its budget on dead updates—then converges only at the end; RLVP rises immediately by learning from the early all-fail groups. Reimplemented dense-process baselines (GiGPO-style step advantages [Feng

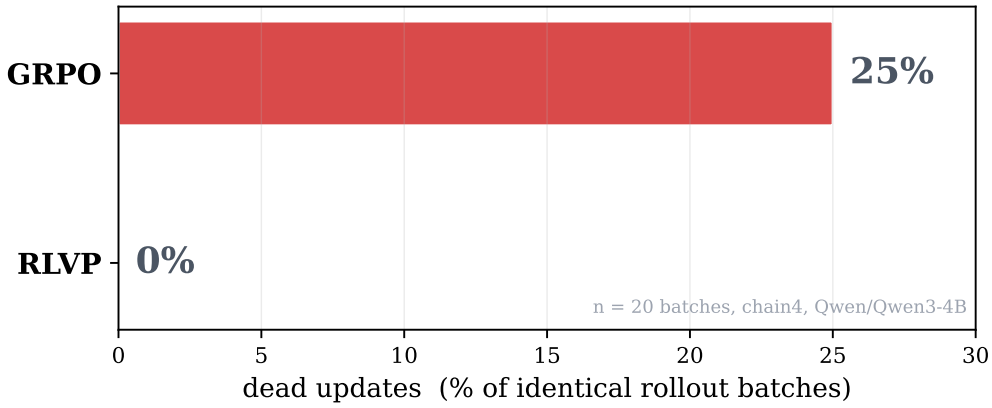

Figure 5. The mechanism, in a paired experiment. One batch of rollouts, scored two ways. Outcome-only credit yields a dead update on one in four identical batches; the verifiable process credit is never dead. The difference is causal—it is the credit scheme, with the data held fixed.

et al., 2025], StepTool-style per-call rewards [Yu et al., 2024]) also far outpace outcome-only: the broad lesson is that *any* admissible dense signal helps. Aggregated over three seeds (Figure 7), RLVP reaches the success threshold several times faster than GRPO with *near-zero* seed variance, while outcome-only GRPO is a lottery governed by when it first stumbles into a non-all-fail group. DAPO [Yu et al., 2025], designed for the same all-fail problem, *discards* such groups and resamples; we measure a 5.6× sampling tax that leaves it, on the fair axis of episodes generated, slower than vanilla GRPO. Confronted with an all-fail group, DAPO says "no information, discard"; RLVP says "no *outcome* information, but plenty of *process* information." Breaking the binary-reward degeneracy is exactly what lets RLVP keep the failures DAPO must throw away.

5 An Anatomy of the Gain

Which component does the work? Ablating on $N{=}4$ (Figure 8) revises some intuitions. **The token-attached channel is the efficiency lever**: folding the same rule rewards into the scalar return doubles the episodes to threshold—placing credit on the responsible tokens, not merely having a dense reward, is what buys the speed, and is the operational reason per-channel (not global) normalization is non-negotiable. **Annealing protects the ceiling**: removing it does not slow learning but lowers final success, because unrelaxed process pressure drifts the policy toward over-compliant, bloated episodes—a mild version of the compliance attractor that returns with force in §7. **The discharge credit is the positive signal that escapes all-fail groups**: penalties suppress wrong moves but cannot induce a never-tried right one, and removing the discharge leaves more dead iterations and a lower ceiling. Finally, **the signal can be free**: replacing every hand-written rule with three *universal* meta-rules derived only from per-tool category tags (observe/mutate/verify/terminal) and the environment's own error codes recovers the hand-engineered efficiency almost exactly, with no task-specific human input—RLVP's specification cost can, in well-structured environments, be paid by metadata that tool schemas already expose.

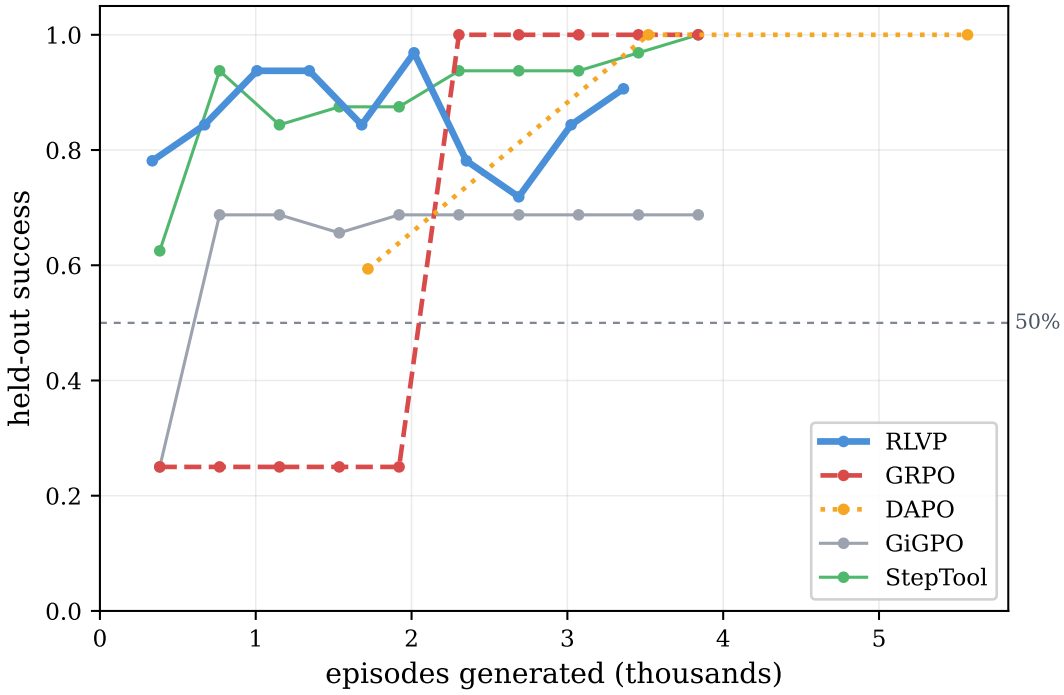

Figure 6. Sample efficiency on the calibrated hard task (N=4): held-out success vs. episodes generated. Outcome-only GRPO sits near the base rate for most of training, then converges only at the end; RLVP climbs immediately by learning from the all-fail groups. Dense-process baselines also far outpace outcome-only. The x-axis counts episodes *generated*, the fair currency that includes DAPO's resampling.

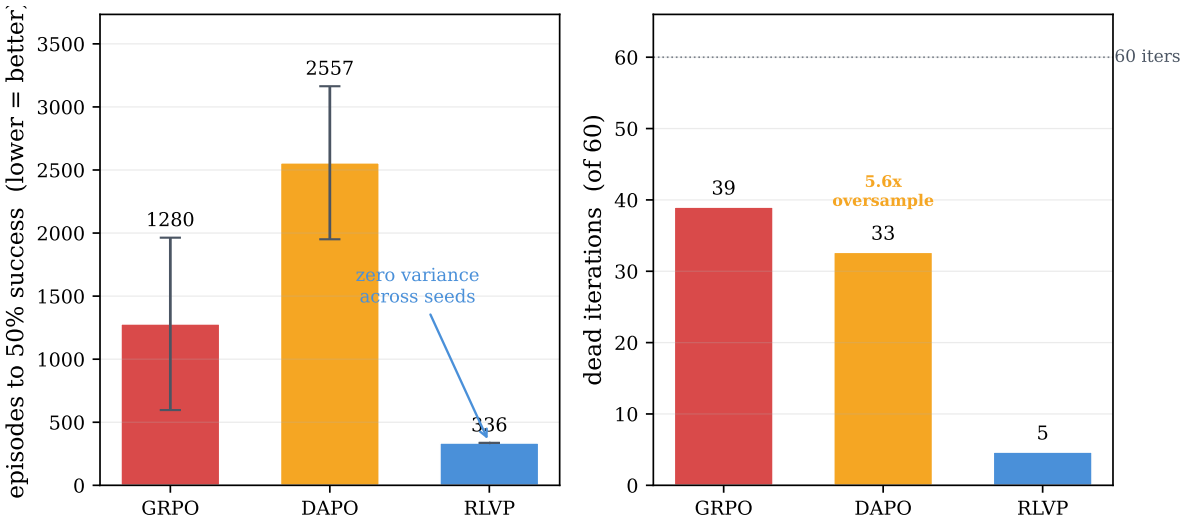

Figure 7. Headline efficiency comparison over three seeds. *Left:* episodes to the success threshold (lower is better)—RLVP is several times faster than GRPO and DAPO and, unlike both, has near-zero seed variance; GRPO's large error bar is the all-fail lottery. *Right:* the mechanism—GRPO and DAPO burn most iterations on dead or stalled updates, and DAPO additionally generates 5.6× the episodes it uses.

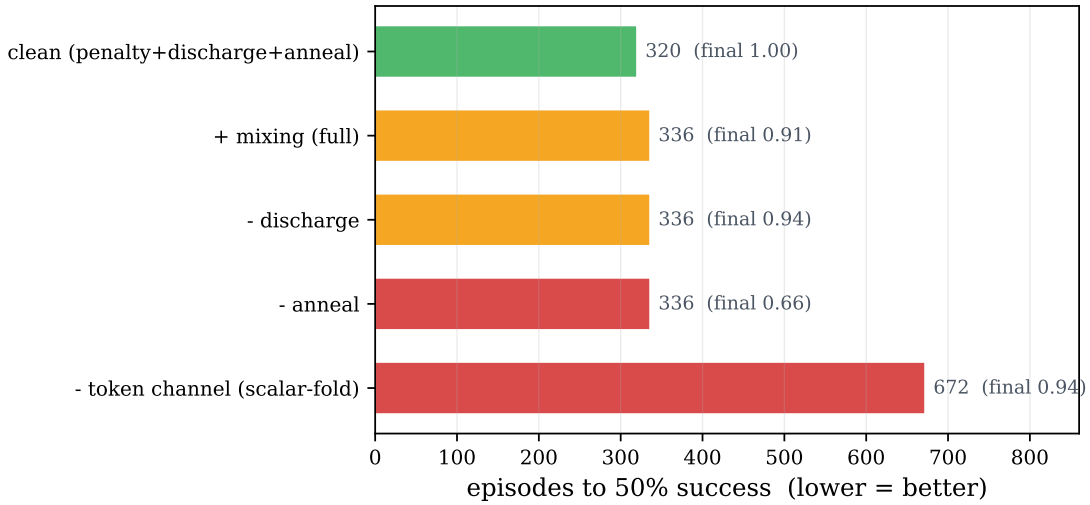

Figure 8. Component attribution on $N{=}4$ (episodes to threshold; final converged success annotated). The token-attached credit channel is the efficiency lever (folding into a scalar return doubles the cost); annealing protects the ceiling; the clean recipe is the best variant. A per-step length cost, helpful only on very long horizons, hurts here.

Table 2. Harm reduction at equal outcome (TerminalBench, Qwen3-4B, 3 seeds; mean $\pm$ std). Task success is identical and at the floor—4B rarely solves the benchmark—yet the un-gameable harm penalty cuts harmful actions roughly fourfold. The policy is not going passive: it takes about three times *more* productive actions while violating less. Harm lives on an axis the terminal outcome cannot see.

	RLVP (harm penalty)	**Outcome-only**
Task success	0.097 ± 0.069	0.097 ± 0.063 (equal, at floor)
Violations / episode	$\mathbf{0.95 \pm 0.71}$	3.99 ± 0.57
Productive actions / episode	$\sim$**14**	$\sim$4

6 Where the Criterion Says Yes, II: Harm Reduction Independent of Outcome

The second place the criterion says yes is the safety use the outcome reward structurally cannot serve. Because the terminal reward is defined on the final state, it is blind to *irreversible harm on the path*; a verifiable penalty on harmful actions is exactly an un-gameable bound on that path (its cheapest maximizer—take the harmful action zero times—*is* the desired behavior, so it does not invite the compliance attractor the way an orthogonal procedural penalty does). We test this on real TerminalBench Docker tasks with Qwen3-4B, with penalties on repeat-error and blind-destructive actions (Table 2). At this scale the model rarely solves the benchmark, so *task success is identical and at the floor*—and yet RLVP commits roughly *four times fewer* harmful actions. Crucially it does not achieve this by going passive: it takes about three times *more* productive actions while violating less. This is the clean separation the criterion predicts—harm lives on an axis independent of the outcome, and an un-gameable penalty reduces it without changing whether the task is solved. (Operationally: you cannot delete the production database and then "finish" the repair—the harm already happened, regardless of the final state.)

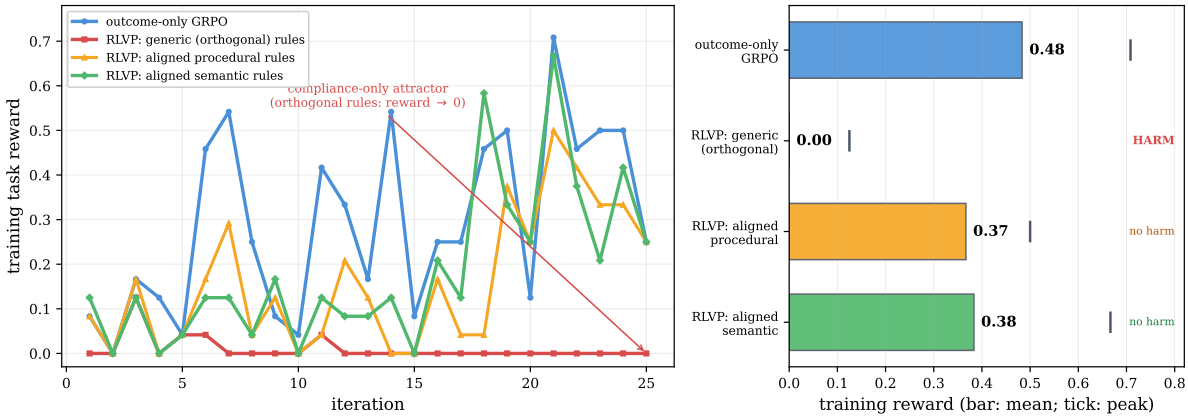

Figure 9. The coverage gradient, and where it saturates, on τ-bench airline. *Left:* training reward for four signals—outcome-only learns the benchmark; generic rules orthogonal to the reward collapse into the compliance attractor; policy-derived procedural and verifiable-semantic rules (with early annealing) do not. *Right:* mean reward per tier (bars) with peak (ticks). Misaligned rules *harm*; aligned rules *remove* the harm and the semantic peak nearly reaches outcome-only's, but neither *beats* it—the residual gap is task intent, which is the reward itself.

7 Where the Criterion Says No, I: Task Intent Is Not a Verifiable Rule

The hardest case for any process-reward method is a task whose difficulty is not procedural at all. We move onto τ-bench-style airline customer service [Yao et al., 2024], whose reward turns on *semantic* policy adherence—authorization, refund eligibility, confirmation—rather than the structural ordering generic rules encode. The criterion predicts the first gate will fail: there is no verifiable potential finer than the outcome, because what is missing is *intent*, which is the reward itself. The experiment (Figure 9) confirms it in three informative tiers. *Generic* structural rules—the read-before-write hygiene that works on chained tasks—are now orthogonal to the reward and actively *harm*: the policy discovers that the cheapest way never to violate "confirm before calling" is to not place the risky calls at all, and collapses into the compliance attractor within a handful of iterations. *Policy-derived procedural* rules (obtain the user id and look up the reservation before modifying it), outcome-instrumental by construction and paired with early annealing, remove the harm—a correct modification *requires* the looked-up state, so the discharge pulls toward the productive workflow rather than idleness. *Verifiable semantic* rules (do not modify a basic-economy reservation; do not use a payment method absent from the profile) prune failure-bound actions and lift the best iterations nearly to outcome-only's peak. But neither aligned tier *beats* outcome-only on average. The coverage gradient saturates: verifiable rules express policy *validity*—whether a modification is permissible—while the residual difficulty is *which* permissible change *this* user wants, and a rule encoding that would simply be the task specification. RLVP accelerates the policy-verifiable component of success and neutralizes harm, but cannot substitute for outcome learning of intent (Figure 10).

8 Where the Criterion Says No, II: A Learned Critic Relocates the Hack

If verifiable rules cannot reach intent, why not let the policy *judge itself*, in the lineage of self-rewarding LMs and RLAIF [Yuan et al., 2024, Lee et al., 2023, Bai et al., 2022, Zheng et al., 2023]? We test this with the critic as the *same model* as the policy (no larger judge), so any

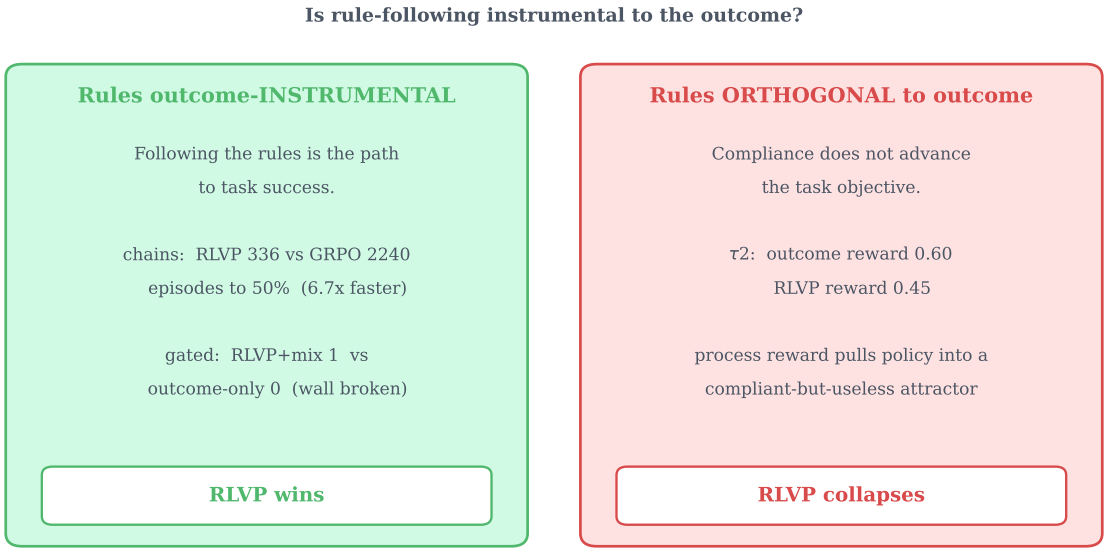

Figure 10. The boundary on a single axis: are the rules outcome-instrumental? When the verifiable rules are aligned with what success requires (chained and gated tasks: the workflow *is* the task), process rewards convert compliance into large outcome gains. When orthogonal to the reward (τ-bench with generic rules), the same machinery optimizes compliance into a degenerate policy.

signal is genuine on-policy self-reflection rather than distillation, mapping outcomes across two axes: whether a verifiable rule is cheaply specifiable, and whether the on-policy critic can detect the violation (Figure 11). As a *detector*, blind self-critique flags violations evident from the trajectory surface but is essentially blind to *stateful-bookkeeping* norms (did it read *this* file before overwriting it) even when handed the rules, and the blind spot is structural—per-rule recall stays near zero as the critic is scaled up. As a *reward* in the all-fail regime (Table 3), a deterministic rule with identical credit structure is decisive on every seed, while the self-critic—*with perfect recall* against the rule oracle—is inert; a *frozen* critic fails identically, isolating the cause as the critic's small but nonzero false-positive rate, not its non-stationarity. At the intent frontier (Table 4), the self-critic is a useful *offline* detector—it far outscores the verifiable rules as a failure predictor—yet *collapses* when used as the training reward. This is the criterion applied to a *learned* signal: defining progress by a learned proxy relocates the credit-assignment problem into the proxy, which is then either too imprecise to optimize against or, at the intent frontier, reduces to approximating the value function itself. Verifiability is not a convenience; it is what makes the dense channel safe to optimize.

9 Where the Criterion Says No, III: The Discovery Ceiling of Self-Evolution

A third boundary is about *reachability* (the criterion's condition (3)), and it is the one that quietly governs hard domains. Sample efficiency is a statement about *speed* to a ceiling the chained tasks eventually reach. To ask whether a process reward can raise the *ceiling*, we need a task the model cannot saturate, and one that is hard to *discover* rather than hard to *compute* (process rewards guide procedure, not answers). We build a *silent precondition gate*: to write a protected file the agent must first read an access-control file and request access; skip either and the write silently no-ops—it reports success but does nothing, and the task fails with no

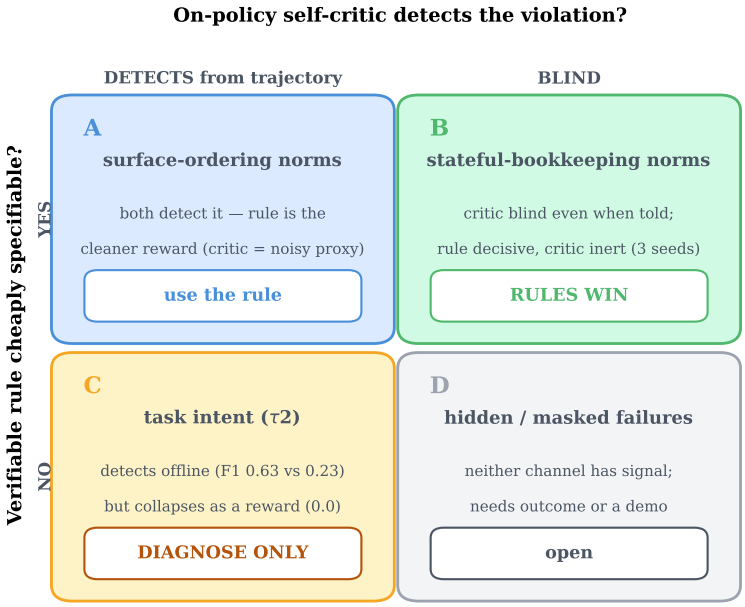

Figure 11. When a verifiable rule beats a same-model self-critic, and when neither trains. Axes: whether a verifiable rule is cheaply specifiable, and whether the on-policy critic can detect the violation. The learned critic is a poor training reward in every quadrant; its only robust use is offline diagnosis at the intent frontier.

Table 3. A deterministic rule beats a same-model self-critic as a dense reward, in the all-fail regime (consumer-service domain, 1.7B, three seeds; late-training mean ± std). Rule and critics share the identical penalty-only credit structure; the critic additionally has *perfect recall* against the rule oracle. The rule is decisive every seed; the critics are inert. The *frozen* (stationary) critic fails like the live one, isolating the cause as the critic's ∼6% false-positive imprecision rather than non-stationarity.

process reward	detection (P/R)	violations/episode	task success
rule penalty (deterministic)	1.00/1.00	$\mathbf{0.38 \pm 0.44}$	$\mathbf{0.33 \pm 0.47}$
self-critic, live (co-evolving)	0.94/1.00	0.94 ± 0.05	0.00
self-critic, frozen (stationary)	0.94/1.00	0.93 ± 0.02	0.00

Table 4. Self-critique is a usable intent *detector* but a failed intent *reward* (τ-bench airline, Qwen3-4B). *Left:* as a failure predictor on rule-clean (intent) failures the self-critic far exceeds the verifiable semantic rules (3 rollout seeds). *Right:* as the dense training reward the same self-critique collapses, while outcome-only and the rules both learn (training reward, early→late; 20 iterations).

offline: failure prediction		*trained as reward*	
channel	F1	channel	reward (early→late)
semantic rules	0.23 ± 0.06	outcome-only	$0.09 \rightarrow \mathbf{0.50}$
blind self-critique	$\mathbf{0.63 \pm 0.02}$	semantic rules	$0.10 \rightarrow 0.35$
		blind self-critique	$0.01 \rightarrow \mathbf{0.00}$

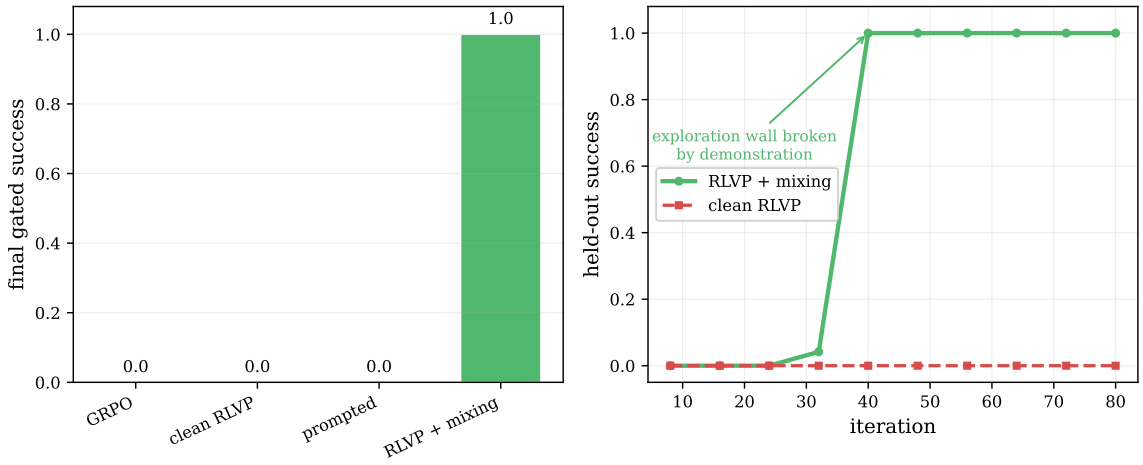

Figure 12. A non-saturating ceiling test, and the discovery wall. *Left:* on the silent-gate task every reward-only method—outcome, DAPO, clean RLVP, even rules-in-prompt—converges to zero, because the required precondition is never sampled and the discharge credit can never fire. *Right:* a single synthesized demonstration injected through the process channel breaks the wall to perfect held-out success—a sharp phase transition, while clean RLVP stays at the floor.

hint why. Calibration confirms the trap is total: the base model never reads the access file, even when the rule is in its prompt. The result (Figure 12) is sharper than "RLVP wins": *every* reward-only method—outcome, DAPO, clean RLVP—fails to zero, because the discharge credit for reading the access file can only reward that behavior if the policy *samples* it, and it never does. A single rule-synthesized demonstration, injected through the process channel, breaks the wall and drives the policy to perfect, generalizing success where prompting the same behavior had failed. When the required behavior is never sampled, no on-policy reward can induce it, and imitation must seed what reward then grows—the same reachability wall the software-repair probe (§3) hit from the other side. It maps cleanly onto R1-Zero (discoverable workflow: reward-only suffices) versus R1 (un-discoverable precondition: a demonstration cold-start is necessary).

10 Discussion

The criterion as a design tool. The contribution is less a method than a decision procedure. Before adding a dense process reward, a practitioner can ask the three questions in turn—does a verifiable potential finer than the outcome exist; is its cheapest maximizer forced to solve the task; do base-policy rollouts reach its intermediate values—and predict the result. The five settings here were not cherry-picked successes; they were chosen to span the gates, and the criterion's verdict matched in every case, including the failures. When the answer to all three is yes, a verifiable progress credit is the right tool and the gains are large; when a penalty is the only available signal, it belongs on the *harm* axis (bounding the path), never as the learning gradient in an all-fail regime; when the residual difficulty is intent, only the outcome can teach it.

A systems note: verifiable agentic RL is often verifier-bound. The property that makes a process reward cheap—a machine-checkable environment—tends to make the verifier a

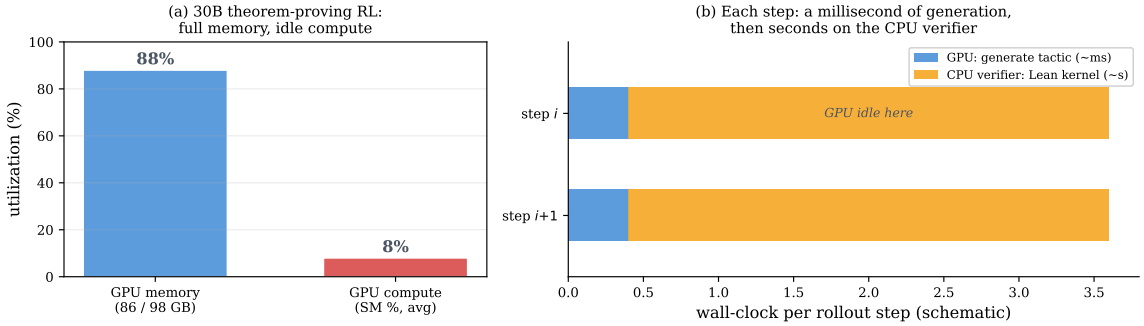

Figure 13. Verifiable agentic RL is often verifier-bound, not compute-bound. *(a)* During 30B theorem-proving RL the GPU runs at full memory but single-digit compute utilization. *(b)* Why: each rollout step spends a millisecond generating a tactic on the GPU, then seconds in the CPU-side Lean kernel verifying it, during which the GPU idles. Throughput then scales with verifier (environment) parallelism rather than accelerators—true for cheap CPU verifiers, though generation-heavy or learned-verifier workloads remain GPU-bound.

CPU process (a proof kernel, a test runner, a container). In our 30B theorem-proving runs the GPU generated a tactic in milliseconds and then waited seconds on the Lean kernel, leaving GPU compute utilization in the single digits at full memory (Figure 13). This is workload-dependent, not a law—generation-heavy or learned-verifier settings are GPU-bound—but for the verifiable domains where our criterion most often says yes, throughput is gated by environment parallelism, and the right systems design co-locates many verifier workers per accelerator rather than scaling accelerators.

Limitations. The 30B un-gameability sweep is high-variance: even over three seeds the per-arm magnitudes carry large error bars (Figure 3), so we report the *survival pattern*—penalty-free survives, pure penalty reliably collapses, penalty-plus-discharge is bimodal—rather than precise values, corroborated by the lower-variance 4B granularity/sparsity study and the τ-bench collapse. The variance itself carries a lesson we did not anticipate: a single seed made the penalty-plus-discharge arm look reliably dead, when three seeds reveal it is occasionally rescued—a caution against reading a single agentic-RL run as a verdict. The reachability result is measured at 30B on a benchmark that is 0%-solve for this model; a stronger model that achieved partial fixes could make the SWE potential non-vacuous, so the claim is explicitly conditional on reachability. And our verifiable potentials are hand-identified per domain; automating their discovery is the natural next step.

11 Related Work

Outcome RL and credit assignment. GRPO [Shao et al., 2024] and the R1 line [DeepSeek-AI, 2025] established outcome-only verifiable-reward RL as the dominant recipe; the long-horizon credit-assignment difficulty has prompted step-level value estimation and turn-level shaping [Feng et al., 2025, Yu et al., 2024]. We frame the problem as the *all-fail-group blindness* of group-relative methods and characterize when a *verifiable, non-learned* process signal fills the gap. DAPO [Yu et al., 2025] is the closest direct comparison; we show its dynamic sampling relocates the all-fail cost rather than recovering the discarded signal.

Process rewards, rubrics, and reward shaping. Process reward models [Lightman et al., 2024] and step-grained tool-use rewards [Yu et al., 2024, Qian et al., 2025] attach credit before the outcome; rubric- and verifier-based rewards [Gunjal et al., 2025] extend verifiable RL beyond auto-checkable domains. Our criterion is orthogonal: it says *when* such a dense signal is admissible. The connection to classical reward shaping is precise but the question is different—potential-based shaping [Ng et al., 1999] certifies that any potential is *safe* (it leaves the optimal policy unchanged), whereas we ask which potential is *useful*, and answer with a property of the group rather than of the optimum: the within-group variance the outcome cannot supply, non-zero (finer, reachable) and aligned (un-gameable).

Learned rewards and self-critique. Self-rewarding LMs [Yuan et al., 2024], RLAIF [Lee et al., 2023], Constitutional AI [Bai et al., 2022], and LLM-as-judge [Zheng et al., 2023] replace rule-based reward with model judgments. Our controlled same-model study finds a learned critic dominated by a deterministic verifiable rule even at perfect recall, and collapsing where no rule applies—evidence that a learned proxy relocates rather than removes reward-hacking.

Demonstrations and the discovery wall. R1-Zero [DeepSeek-AI, 2025] self-evolves with no cold-start; off-policy guidance mixes expert trajectories into on-policy RL [Yan et al., 2025]. We locate the boundary empirically: reward-only suffices on discoverable workflows, and a demonstration becomes necessary precisely when a required behavior is never sampled. Customer-service benchmarks with explicit policy documents [Yao et al., 2024] motivate our intent-ceiling analysis.

12 Conclusion

Agentic RL's environment is a verifier, offering dense intermediate signal that reasoning RL never had—but using it naively either wastes it (a uniform signal centered out by group-relative RL) or is catastrophic (a gameable penalty's compliance collapse, a hacked learned critic). We gave a single criterion that resolves when the signal helps: a dense process reward improves agentic RL if and only if it is an un-gameable, verifiable potential finer than the outcome whose intermediate values the policy can reach. The criterion is checkable before training, grounded in potential-based shaping, and predictive: across five settings it called every success and every failure, including a controlled sweep in which penalty kills and outcome-gating rescues exactly as predicted. Where it says yes, the payoff is concrete—dead updates eliminated and harm cut fourfold at equal success; where it says no, it says so for a reason—intent is not a verifiable rule, and a structurally-finer potential is worthless if unreachable. The durable contribution is not another process-reward method but the line the criterion draws—between the part of agentic success that a verifiable signal can densify and accelerate, and the part that only the outcome can teach—and the one quantity behind it: a useful dense process reward is the within-group variance the outcome cannot supply.

References

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Lang Feng et al. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025.

Anisha Gunjal et al. Rubrics as rewards: Reinforcement learning beyond verifiable domains. *arXiv preprint arXiv:2507.17746*, 2025.

Dingkun Hong. AI Coding: Practice and exploration, 2026. Keynote, ByteDance / Volcano Engine Force Conference, Beijing, June 2026. https://mp.weixin.qq.com/s/tJAinVKzZAqZrhiEvGU2Dg (accessed 2026-06-25).

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Harrison Lee, Samrat Phatale, Hassan Mansoor, Kellie Lu, Thomas Mesnard, Colton Bishop, Victor Carbune, and Abhinav Rastogi. RLAIF: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2023.

Hunter Lightman, Vineet Kosaraju, Yura Burda, et al. Let's verify step by step. *International Conference on Learning Representations (ICLR)*, 2024.

Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

Cheng Qian et al. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Jianhao Yan et al. Learning to reason under off-policy guidance. *arXiv preprint arXiv:2504.14945*, 2025.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Yuanqing Yu et al. Steptool: Enhancing multi-step tool usage in llms through step-grained reinforcement learning. *arXiv preprint arXiv:2410.07745*, 2024.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. minif2f: a cross-system benchmark for formal olympiad-level mathematics. *International Conference on Learning Representations (ICLR)*, 2022.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023.